S.No: 21	Exp. Name: *Tuple Creation*	Date:2026-10-02

Aim:

Write a Python program that prompts the user to enter details for two members and creates tuples for each member consisting of: name (string), age (int), address (string), college (string). Then, concatenate these two tuples into a single tuple and print all three tuples.

Input Format:

- The first line contains four space-separated values for Member 1:
  - name
  - age
  - address
  - college
- The second line contains the details of Member 2 in the same format

Output Format:

- Print the tuple for Member 1.
- Print the tuple for Member 2.
- Print the concatenated tuple of both members.

Sample Test Case:

Input:

```
John 12 Tampa St.Joseph
Jane 15 Florida St.Mary
```

Output:

```
('John', 12, 'Tampa', 'St.Joseph') # Member 1
('Jane', 15, 'Florida', 'St.Mary') # Member 2
('John', 12, 'Tampa', 'St.Joseph', 'Jane', '15', 'Florida', 'St.Mary') # Concatenated Tupl
```

Source Code:

stdTup.py

```
name1, age1, address1, college1= input().split()
member1 =(name1, int(age1), address1, college1)
name2, age2, address2, college2 = input().split()
member2 = (name2, int(age2),address2, college2)
concatenated_tuple = member1+member2
print(member1)
print(member2)
print(concatenated_tuple)
```


Execution Results - All test cases have succeeded!

Test Case - 1
User Output
John 12 Tampa St.Joseph
Jane 15 Florida St.Mary

('John', 12, 'Tampa', 'St.Joseph')
('Jane', 15, 'Florida', 'St.Mary')
('John', 12, 'Tampa', 'St.Joseph', 'Jane', 15, 'Florida', 'St.Mary')

Test Case - 2
User Output
Ram 22 Delhi IITDelhi
Varun 24 Mumbai NITWarangal
('Ram', 22, 'Delhi', 'IITDelhi')
('Varun', 24, 'Mumbai', 'NITWarangal')
('Ram', 22, 'Delhi', 'IITDelhi', 'Varun', 24, 'Mumbai', 'NITWarangal')

S.No: 22	Exp. Name: *Vowel Count without using Control Flow Statements*	Date:2026-10-02

Aim:

Write a Python program to count the number of vowels in a given string without using any control flow statements (i.e., do not use if, for, while, or any conditional/loop constructs).

Input Format:

- A single line containing a string s.

Output Format:

- Print the number of vowels (both uppercase and lowercase) in the string.

Source Code:

vowelCount.py

```
s=input()
count=(s.lower().count('a')+s.lower().count('e')+s.lower().count('i')+s.lower
().count('o')+s.lower().count('u'))
print(count)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Education is divine
9

Test Case - 2
User Output
Rhythm
0

S.No: 23	Exp. Name: ***Key Lookup in Dictionary***	Date:2026-10-02

Aim:

Write a Python program that checks if a given key exists in a dictionary.

Input Format:

- The first line of input contains space separated strings that represent the keys of a dictionary.
- The second line contains space separated strings that represent the values of a dictionary.
- If the number of keys and values are not equal, the program should immediately print "Invalid" and terminate without taking the third input.
- Otherwise, the third line contains a string representing the key to check.

Output Format:

- If the number of keys and values are not equal, print "Invalid".
- Otherwise:
  Print the dictionary in the first line.
  Print "Exist" if the key is present in the dictionary.
  Print "Does not Exist" if the key is not present in the dictionary.

Note:

- Refer to the visible test cases to strictly match the input/output format.

Source Code:

keyExist.py

```
keys=input().split()
values=input().split()
if len(keys)!=len(values):
        print("Invalid")
else:
        d=dict(zip(keys, values))
        key=input()
        print(d)
        if key in d:
                print("Exist")
        else:
                print("Does not Exist")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 2 3 4 5
A B C D E
5
{'1': 'A', '2': 'B', '3': 'C', '4': 'D', '5': 'E'}
Exist

Test Case - 2

User Output
10 20 30 40 50 60 70 80
Ten Twenty Thirty Forty Fifty Sixty
Invalid

Test Case - 3
User Output
John Reena Sara David
24 32 18 41
Jane
{'John': '24', 'Reena': '32', 'Sara': '18', 'David': '41'}
Does not Exist

S.No: 24	Exp. Name: *Add Key Value Pair to Dictionary*	Date:2026-10-02

Aim:

Write a Python program that adds a new key-value pair to an existing dictionary.

Input Format:

- The first line contains space-separated strings that represent the keys of a dictionary.
- The second line contains space-separated strings that represent the values of the dictionary.
- The third line contains two space-separated strings for the new key and value,only if the number of keys and values match; otherwise, print "Invalid" and skip this input.

Output Format:

- First line should display the original dictionary before adding the new key-value pair.
- Second line should display the updated dictionary after adding the new key-value pair.

Note:

- If the number of keys and values does not match, display "Invalid" without taking the key-value pair as input.

Source Code:

keyValue.py

```
keys=input().split()
values=input().split()
if len(keys)!= len(values):
        print("Invalid")
else:
        d=dict(zip(keys,values))
        new_key,new_value=input().split()
        print(d)
        d[new_key]=new_value
        print(d)
```


Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Mobile Laptop Adopter Headset
10000 50000 2000
Invalid

Test Case - 2
User Output
thirty fifty seventy ninety forty eighty
30 50 70 90 40 80
four 4
{'thirty': '30', 'fifty': '50', 'seventy': '70', 'ninety': '90', 'forty': '40', 'ei
{'thirty': '30', 'fifty': '50', 'seventy': '70', 'ninety': '90', 'forty': '40', 'ei

S.No: 25	Exp. Name: *Sum of all Items in a Dictionary*	Date:2026-10-02

Aim:

Write a program that allows the user to create a dictionary by entering key-value pairs. After creating the dictionary, calculate and print the sum of all values in it.

Input Format:

1. The first line prompts "size:" followed by an integer $size$, representing the number of key-value pairs to be added to the dictionary.
2. For each key-value pair:
   - The first line prompts "key:" followed by a string representing the key on the same line.
   - The second line prompts "value:" followed by an integer representing the $value$ on the same line.

Output Format:

- Print "Sum:" followed by the total sum of all values without any space on the same line.

Note:

- Refer to the visible test cases for better understanding of input and output format.

Source Code:

sumOfValues.py

```
n=int(input("size:"))
d={}
for i in range(n):
        key=input("key:")
        value=int(input("value:"))
        d[key]=value
        total=sum(d.values())
print("Sum:",total,sep="")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
size:
1
key:
ninety
value:
90
Sum:90

Test Case - 2
User Output
size:
4
key:

five
value:
5
key:
six
value:
6
key:
twelve
value:
12
key:
sixty
value:
60
Sum:83

S.No: 26	Exp. Name: *Copy File contents to another File with Lowered Characters*	Date:2026-10-04

Aim:

Write a Python program that reads a text file, extracts all the words, converts them to lowercase, and writes them to another file. After writing, display the contents of the output file. Always assume that the input file exists.

Input Format:

- The name of the input file (e.g., file1.txt) will be provided by the user.
- The name of the output file (e.g., hiddenfile1.txt) will also be provided by the user.

Output Format:

- The output file should contain all the lowercase words from the input file, one word per line, in the same order.
- The program should then display the contents of the output file on the screen.

Source Code:

CopyFileContents.py

```
input_file=input()
output_file=input()
with open(input_file, "r") as f:
        words=f.read().split()
with open(output_file, "w")as f:
        for word in words:
                f.write(word.lower()+"\n")
with open(output_file,"r")as f:
        print(f.read(),end="")
```

file1.txt

```
Welcome to CodeTantra
Start coding in 60min.
```

file2.txt

```
This is a sample text file.
It contains multiple lines.
Each line has different words.
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
file1.txt
output.txt
welcome

to
codetantra
start
coding
in
60min.

Test Case - 2
User Output
file2.txt
output1.txt
this
is
a
sample
text
file.
it
contains
multiple
lines.
each
line
has
different
words.

S.No: 27	Exp. Name: ***File Reverse***	Date:2026-10-04

Aim:

Sara is developing a tool that manipulates text files efficiently. Your task is to assist Sara in creating a Python program that reads a text file and prints the contents of each line in reverse order.

Input Format:

- The program should prompt the user to enter the file name.

Output Format:

- If the entered file is found, then it should display the contents of each line of the file in reverse order; otherwise, "File not found".

Source Code:

reverse.py

```
a = input()
try:
        f = open(a, "r")
        for line in f:
                print(line.strip()[::-1])
        f.close()
except:
        print("File not found")
```

file1.txt

```
Insert
Here
Text
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
output.txt
File not found

Test Case - 2
User Output
file1.txt
tresnI
ereH
txeT

S.No: 28	Exp. Name: *Count Characters, Words, and Lines in a File*	Date:2026-10-04

Aim:

You need to write a Python program that computes the number of characters, words, and lines in a file.

- **Characters**: The number of characters (including spaces and punctuation) in the file.
- **Words**: The number of words in the file (separated by spaces).
- **Lines**: The number of lines in the file.

Input Format:

- The input line reads a string representing the file name.

Output Format:

- The output lines contain integers representing the count of characters, words, and lines each on a new line.

Source Code:

Counting.py

```
a = input()
f = open(a, "r")
data = f.read()
f.close()
print(len(data))
print(len(data.split()))
print(len(data.splitlines()))
```

file1.txt

```
This is the introduction.
We are testing a file comparison program.
It checks line-by-line differences.
The program is quite useful.
End of the file.
```

file2.txt

```
The quick brown fox jumps over the lazy dog.
A quick red fox jumps over a sleepy dog.
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
file1.txt
149
24

5

Test Case - 2
User Output
file2.txt
85
18
2

S.No: 29	Exp. Name: *Array Operations*	Date:2026-10-05

Aim:

Write a Python program to create an array of strings entered by the user and perform the following operations sequentially:

- Display the array.
- Append an item to the array.
- Insert an item at a specified position in the array.
- Reverse the order of items in the array.

Input Format:

- The first line contains space-separated string values to initialize the array.
- The second line contains a single string to append to the array.
- The third line contains an integer representing the position at which to insert the value.
- The fourth line contains a single string value to insert into the array.

Output Format:

- Display the created array.
- Display the array after appending the value.
- Display the array after inserting the value at the specified position.
- Display the reversed array.

Note: The index position to insert the value should be less than or equal to the length of the array. Otherwise, print "Invalid index".

Refer to the shown test cases to match the input and output format with the test cases.

Source Code:

arrayOp.py

```
arr=input().split()
append_val=input()
index=int(input())
insert_val=input()
print(arr)
arr.append(append_val)
print(arr)
if index<=len(arr)or index<0:
        arr.insert(index, insert_val)
        print(arr)
else:
        print("Invalid index")
arr.reverse()
print(arr)
```


Execution Results - All test cases have succeeded!

Test Case - 1
User Output
12 23 34 45 56 67

85
2
69
['12', '23', '34', '45', '56', '67']
['12', '23', '34', '45', '56', '67', '85']
['12', '23', '69', '34', '45', '56', '67', '85']
['85', '67', '56', '45', '34', '69', '23', '12']

Test Case - 2
User Output
a b c d e f g h
k
5
o
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'k']
['a', 'b', 'c', 'd', 'e', 'o', 'f', 'g', 'h', 'k']
['k', 'h', 'g', 'f', 'o', 'e', 'd', 'c', 'b', 'a']

Test Case - 3
User Output
hi hello how are you
fine
7
you
['hi', 'hello', 'how', 'are', 'you']
['hi', 'hello', 'how', 'are', 'you', 'fine']
Invalid index
['fine', 'you', 'are', 'how', 'hello', 'hi']

S.No: 30	Exp. Name: ***Matrix Addition***	Date:2026-10-05

Aim:

Write a Python program to add two matrices. The program reads the dimensions and elements of two matrices and computes their sum if the dimensions are compatible. If the dimensions are incompatible, the program prints an appropriate error message.

Two matrices can be added only if they have the same number of rows and columns, i.e., $r1 = r2$ and $c1 = c2$.

Input Format:

- The first line contains two space-separated integers representing the number of rows ($r1$) and columns($c1$) for matrix1
- Next, $r1$ lines contain $c1$ number of integers separated by a space, representing the elements of matrix 1
- The next line contains two space-separated integers representing the number of rows ($r2$) and columns ($c2$) for matrix2
- Next, $r2$ lines contain $c2$ number of integers separated by a space, representing the elements of matrix 2

Output Format:

- If the matrices have compatible dimensions ($r1 == r2$ and $c1 == c2$), print the resulting matrix where each row is printed on a new line and elements are separated by spaces, otherwise print "Invalid".

Source Code:

matrix.py

```
r1,c1 = map(int,(input().split()))
matrix = []
for i in range(r1):
        matrix.append(list(map(int,input().split())))
r2,c2 = map(int,input().split())
matrix2 = []
for i in range(r2):
        matrix2.append(list(map(int,input().split())))

if r1 == r2 and c1 == c2:
        for i in range(r1):
                result = []
                for j in range(c1):
                        result.append(matrix[i][j] + matrix2[i][j])
                print(*result)
else:
        print("Invalid")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2 3

1 2 3
4 5 6
2 3
7 8 9
0 1 2
8 10 12
4 6 8

Test Case - 2
User Output
2 2
1 2
2 3
2 3
1 2 3
0 2 3
Invalid

S.No: 31	Exp. Name: ***Matrix Multiplication***	Date:2026-10-05

Aim:

Write a Python function **matrix_multiplication()** that performs matrix multiplication of two matrices and prints the resulting matrix. If the matrices cannot be multiplied, the function should print an appropriate error message.

Input Format:

- The dimensions of the first matrix ($r1$ $c1$), where $r1$ is the number of rows and $c1$ is the number of columns.
- The elements of the first matrix are provided row by row.
- The dimensions of the second matrix ($r2$ $c2$), where $r2$ is the number of rows and $c2$ is the number of columns.
- The elements of the second matrix are provided row by row.

Output Format:

- Print the resulting matrix after multiplication, where each element of the matrix is separated by a space; otherwise, print "Invalid".

Note: The number of columns of the first matrix must be equal to the number of rows of the second matrix for matrix multiplication.

Source Code:

matrixmul.py

```
# define the function
def matrix_multiplication():
        r1,c1 = map(int,input().split())
        matrix1=[list(map(int,input().split())) for _ in range(r1)]
        r2,c2=map(int,input().split())
        matrix2=[list(map(int, input().split())) for _ in range(r2)]
        if c1 != r2:
                print("Invalid")
                return
        result=[[0] * c2 for _ in range(r1)]
        for i in range(r1):
                for j in range(c2):
                        for k in range(c1):
                                result[i][j] += matrix1[i][k] * matrix2[k][j]
        for row in result:
                        print(*row)
if __name__ == "__main__":
                matrix_multiplication()
```


Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2 3
1 2 3

4 5 6
3 4
1 2 3 4
5 6 7 8
9 0 1 2
38 14 20 26
83 38 53 68

Test Case - 2
User Output
2 2
-3 4
4 -7
2 2
-4 5
-6 7
-12 13
26 -29

Test Case - 3
User Output
2 2
6 12
-3 7
3 2
12 23 34
12 34 54
4 26 19
Invalid

S.No: 32	Exp. Name: *Transpose of Matrix*	Date:2026-10-05

Aim:

Write a Python program to find the transpose of a matrix.

Input Format:

- The first line contains two space-separated integers representing the number of rows (R) and columns (C) for the matrix.
- Next, R lines contain C number of integers separated by a space, representing the elements of the matrix.

Output Format:

- Print the transpose of the matrix, with each row on a new line and elements space-separated.

Source Code:

transpose.py

```
r, c = map(int, input().split())
matrix=[list(map(int, input().split())) for _ in range(r)]
transpose=[list(row) for row in zip(*matrix)]
for row in transpose:
        print(*row)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2 4
10 20 30 40
12 23 45 52
10 12
20 23
30 45
40 52

Test Case - 2
User Output
3 5
-10 -20 -30 -40 -50
-11 -22 -33 -44 -55
-13 -15 -17 -18 -19
-10 -11 -13
-20 -22 -15
-30 -33 -17
-40 -44 -18
-50 -55 -19

S.No: 33	Exp. Name: *Geometry Class*	Date:2026-10-05

Aim:

Write a Python program that defines a base class **Shape** with two methods, **area** and **perimeter**, that are meant to be overridden by subclasses. Define three subclasses: **Circle**, **Triangle**, and **Square**, each of which implements the **area** and **perimeter** methods. The program should then create instances of these subclasses based on user input and print their areas and perimeters, formatted to two decimal places.

Input Format:

- The first line contains a single float representing the radius of a circle.
- The second line contains three space-separated floats representing the sides of a triangle.
- The third line contains a single float representing the side length of a square.

Output Format:

- Print the area and perimeter of the circle on one line, separated by a space, formatted to two decimal places.
- Print the area and perimeter of the triangle on the next line, separated by a space, formatted to two decimal places.
- Print the area and perimeter of the square on the last line, separated by a space, formatted to two decimal places.

Note:

- Triangle sides must satisfy the triangle inequality theorem: the sum of any two sides must be greater than the third side, ensuring the formation of a valid triangle.
- Area of traingle $ABC = \sqrt{[s \times (s-a) \times (s-b) \times (s-c)]}$
- $s = \frac{(side1 + side2 + side3)}{2}$

Source Code:

geometry1.py

```
import math
class Shape:
        def area(self):
                pass
        def perimeter(self):
                        pass

class Circle(Shape):
        def __init__(self, radius):
                self.radius=radius
        def area(self):
                return math.pi*(self.radius**2)
        def perimeter(self):
                return 2*math.pi*self.radius

class Triangle(Shape):
        def __init__(self, a ,b, c):
                self.a=a
                self.b=b
                self.c=c
        def area(self):
                s=(self.a+self.b+self.c)/2
```

```
                return math.sqrt(s*(s-self.a)*(s-self.b)*(s-self.c))
        def perimeter(self):
                return self.a+self.b+self.c
class Square(Shape):
        def __init__(self, side):
                self.side=side
        def area(self):
                return self.side**2
        def perimeter(self):
                return 4*self.side

if __name__=="__main__":
        radius=float(input())
        a, b, c=map(float, input().split())
        side=float(input())
        circle=Circle(radius)
        triangle=Triangle(a,b,c)
        square=Square(side)
        print(f"{circle.area():.2f} {circle.perimeter():.2f}")
        print(f"{triangle.area():.2f} {triangle.perimeter():.2f}")
        print(f"{square.area():.2f} {square.perimeter():.2f}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
12
4 5 6
4
452.39 75.40
9.92 15.00
16.00 16.00

Test Case - 2
User Output
5.0
3.0 4.0 5.0
7.5
78.54 31.42
6.00 12.00
56.25 30.00

S.No: 34	Exp. Name: *Check for Complex JSON Object in the String*	Date:2026-10-03

Aim:

Write a Python program to check whether a given JSON string represents a complex object (a dictionary or a list).

Define a function **check_json_complexity(json_string)** that:

- Parses the JSON string using **json.loads()**.
- Checks whether the parsed JSON is a dictionary or a list.
- Returns True if it is a dictionary or a list; otherwise, returns False.

Source Code:

jsonString.py

```
import json
def check_json_complexity(json_string):
        data=json.loads(json_string)
        return isinstance(data,(dict, list))

s=input()
if check_json_complexity(s):
        print("Yes")
else:
        print("No")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
{"name": "Alice", "age": "twenty-five"}
Yes

Test Case - 2
User Output
"Hello, World!"
No

Test Case - 3
User Output
{"name": "Bob", "hobbies": ["reading", "gaming"]}
Yes

S.No: 35	Exp. Name: *Create NumPy Array*	Date:2026-10-03

Aim:

You are given a Python program that takes two integer inputs: a start value and an end value. Write a Python program that uses these inputs to create a NumPy array containing values ranging from the start value to the end value, inclusive. Display the array.

Input Format:

- The first integer represents the start value.
- The second integer represents the end value.

Output Format:

- The program should output the created NumPy array.

Source Code:

createArray.py

```
# Write your code here...
import numpy as np;
num=int(input())
num1=int (input())
my_array=np.arange(num,num1+1)
print(my_array)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10
50
[10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50]

Test Case - 2
User Output
4
9
[4 5 6 7 8 9]

S.No: 36	Exp. Name: ***Numpy Array Operations***	Date:2026-10-03

Aim:

Write a Python program using NumPy to create an array and demonstrate the following:

- Number of dimensions (ndim)
- Shape of the array (shape)
- Total number of elements (size)
- Data type of elements (dtype)

Input Format:

- A single line containing space-separated integers.

Output Format:

Print the following details in the given order:

- The NumPy array
- Number of dimensions
- Shape of the array
- Total number of elements
- Data type of the array

Note: Refer to the visible test-cases for better understanding.

Source Code:

numpyArray.py

```
import numpy as np
a=np.array(list(map(int,input().split())))
print(a)
print(a.ndim)
print(a.shape)
print(a.size)
print(a.dtype)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 2 3 4 5 6
[1 2 3 4 5 6]
1
(6,)
6
int32

Test Case - 2
User Output
10 20 30 40 50 60 70
[10 20 30 40 50 60 70]
1

(7,)
7
int32

S.No: 37	Exp. Name: *Slicing and Indexing*	Date:2026-10-03

Aim:

The given code creates a 3x3 NumPy array and prints the entire array along with the text "Accessing" and "Slicing".

You need to complete the program to demonstrate:

Indexing:

- Read two space-separated integers from the user representing the row index and column index.
- Print the element located at the specified row and column.

Slicing:

- Read two space-separated integers representing the start row index and end row index.
- Read two space-separated integers representing the start column index and end column index.
- Print the sliced subarray using the specified row and column ranges.

Input/Output Formats:

- The predefined 3 × 3 NumPy array is displayed before reading any input.
- Print the heading:

```
Accessing
```

- The next line contains two space-separated integers representing the row index and column index for accessing an element.
- Print the element at the specified row and column index.
- Print the heading:

```
Slicing
```

- The next line contains two space-separated integers representing the start row index and end row index for slicing.
- The following line contains two space-separated integers representing the start column index and end column index for slicing.
- Print the sliced subarray corresponding to the specified row and column ranges.

Note:

- Assume all inputs are valid and within the bounds of the array.
- An array has been provided to you in the editor.

Source Code:

sliceIndex.py

```
import numpy as np

# Create a 3x3 NumPy array
array = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])
print(array)
print("Accessing")
r,c=map(int,input().split())
print(array[r,c])
print("Slicing")
```

```
r1,r2=map(int,input().split())
c1,c2=map(int,input().split())
print(array[r1:r2,c1:c2])
```


Execution Results - All test cases have succeeded!

Test Case - 1
User Output
[[1 2 3]
[4 5 6]
[7 8 9]]
Accessing
1 0
4
Slicing
1 2
1 3
[[5 6]]

Test Case - 2
User Output
[[1 2 3]
[4 5 6]
[7 8 9]]
Accessing
2 2
9
Slicing
0 3
0 3
[[1 2 3]
[4 5 6]
[7 8 9]]

Test Case - 3
User Output
[[1 2 3]
[4 5 6]
[7 8 9]]
Accessing
0 1
2
Slicing
1 2
2 1
[]

S.No: 38	Exp. Name: *Numpy Array Statistics*	Date:2026-10-04

Aim:

Write a Python program that prompts the user to enter elements for a NumPy array. The program should compute and print the minimum value, maximum value and sum of elements of the array using NumPy functions.

Input Format:

- The program expects the user to enter elements of the array separated by spaces.

Output Format:

- First, print the original NumPy array as integers.
- Next, print the minimum value found in the array as an integer.
- Then, display the maximum value present in the array as an integer.
- Finally, show the sum of all elements within the array as an integer.

Source Code:

arrayStat.py

```
import numpy as np
elements=list(map(int,input().split()))
arr=np.array(elements,dtype=int)
min_val=np.min(arr)
max_val=np.max(arr)
array_sum=np.sum(arr)
print(arr)
print(min_val)
print(max_val)
print(array_sum)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 2 3 4 5 6 7 8 9
[1 2 3 4 5 6 7 8 9]
1
9
45

Test Case - 2
User Output
-10 -20 -30 -40 -50
[-10 -20 -30 -40 -50]
-50
-10
-150

S.No: 39	Exp. Name: *Median, Cumulative Sum, and Cumulative Product using NumPy*	Date:2026-10-04

Aim:

Write a Python program that calculates and displays the Median, Cumulative Sum, and Cumulative Product for a given list of numbers using the NumPy library. The program should take a list of integers as input from the user, separated by spaces, and output the results.

Input Format:

- A single line of input contains integers separated by space.

Output Format:

- Print the median rounded to two decimal places, cumulative sum and cumulative product, each in a line respectively.

Source Code:

AggregateFunc.py

```
import numpy as np
numbers=list(map(int,input().split()))
arr=np.array(numbers)
median_val=np.median(arr)
cum_sum=np.cumsum(arr)
cum_prod=np.cumprod(arr)
print(f"{median_val:.1f}")
print(cum_sum)
print(cum_prod)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 2 3 4 5
3.0
[1 3 6 10 15]
[1   2   6 24 120]

Test Case - 2
User Output
3 6 9 7 1
6.0
[3 9 18 25 26]
[3   18 162 1134 1134]

S.No: 40	Exp. Name: *Working with Pandas DataFrame*	Date:2026-10-04

Aim:

A dictionary containing data is provided by default in the editor. Your task is to:

1. Convert the given dictionary into a pandas DataFrame.
2. Perform the following operations exactly as mentioned below:
   - Display the first 5 rows of the DataFrame using the **head()** function.
   - Select and display only the **Name** and **Salary** columns.
   - Display all rows where **Age** is greater than 25.
   - Display all rows where **Department** is **IT**.
   - Display all rows where both **Age** is greater than 25 and **Salary** is greater than 55000.

Source Code:

PandasOperations.py

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace', 'Han
nah', 'Ian', 'Julia'],
    'Age': [25, 30, 22, 28, 24, 27, 29, 23, 26, 31],
    'Department': ['HR', 'Finance', 'IT', 'HR', 'Finance', 'IT', 'HR', 'Financ
e', 'IT', 'HR'],
    'Salary': [50000, 60000, 55000, 52000, 61000, 53000, 58000, 62000, 54000,
57000],
    'Experience': [2, 4, 1, 3, 2, 2, 5, 4, 3, 6]
}
df=pd.DataFrame(data)
# Step 2: Perform the following operations exactly as instructed
# a) Display the first 5 rows of the DataFrame
print(df.head())
# b) Select and display only the 'Name' and 'Salary' columns
print(df[['Name', 'Salary']])
# c) Display all rows where 'Age' is greater than 25
print(df[df['Age']>25])
# d) Display all rows where 'Department' is 'IT'
print(df[df['Department']=='IT'])
# e) Display all rows where both 'Age' > 25 and 'Salary' > 55000
print(df[(df['Age']>25)&(df['Salary']>55000)])
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output

```
      Name  Age Department  Salary  Experience
0    Alice   25         HR   50000           2
1      Bob   30    Finance   60000           4
2  Charlie   22         IT   55000           1
3    David   28         HR   52000           3
4      Eva   24    Finance   61000           2
      Name  Salary
```

0	Alice	50000
1	Bob	60000
2	Charlie	55000
3	David	52000
4	Eva	61000
5	Frank	53000
6	Grace	58000
7	Hannah	62000
8	Ian	54000
9	Julia	57000

	Name	Age	Department	Salary	Experience
1	Bob	30	Finance	60000	4
3	David	28	HR	52000	3
5	Frank	27	IT	53000	2
6	Grace	29	HR	58000	5
8	Ian	26	IT	54000	3
9	Julia	31	HR	57000	6

	Name	Age	Department	Salary	Experience
2	Charlie	22	IT	55000	1
5	Frank	27	IT	53000	2
8	Ian	26	IT	54000	3

	Name	Age	Department	Salary	Experience
1	Bob	30	Finance	60000	4
6	Grace	29	HR	58000	5
9	Julia	31	HR	57000	6

S.No: 41	Exp. Name: *Scatter Plot using Pandas Dataframe*	Date:2026-10-04

Aim:

Create a scatter plot to visualize the relationship between two columns from a dataset represented as a dictionary. Convert the dictionary into a Pandas DataFrame for this purpose.

Input Format:

- Prompt the user to input two column names (strings) separated by space.

Output Format:

- Display a scatter plot showing the relationship between the selected columns.
- Ensure the plot includes appropriate labels, title, legend, grid, and customization (such as color and marker style).

Plot Settings:

- Use blue color to set the color of the markers
- Marker Style: Use marker='o' to set the marker style
- Use "Scatter" as label for the scatter plot in the legend.
- Set "Scatter Plot" as the title of the plot.
- Use column1 and column2 as the labels for the x-axis and y-axis, respectively.
- Display the legend.
- Display the grid on the plot for better visualization.

Note: Refer to the data set that has been provided to you in the editor.

Source Code:

scatter.py

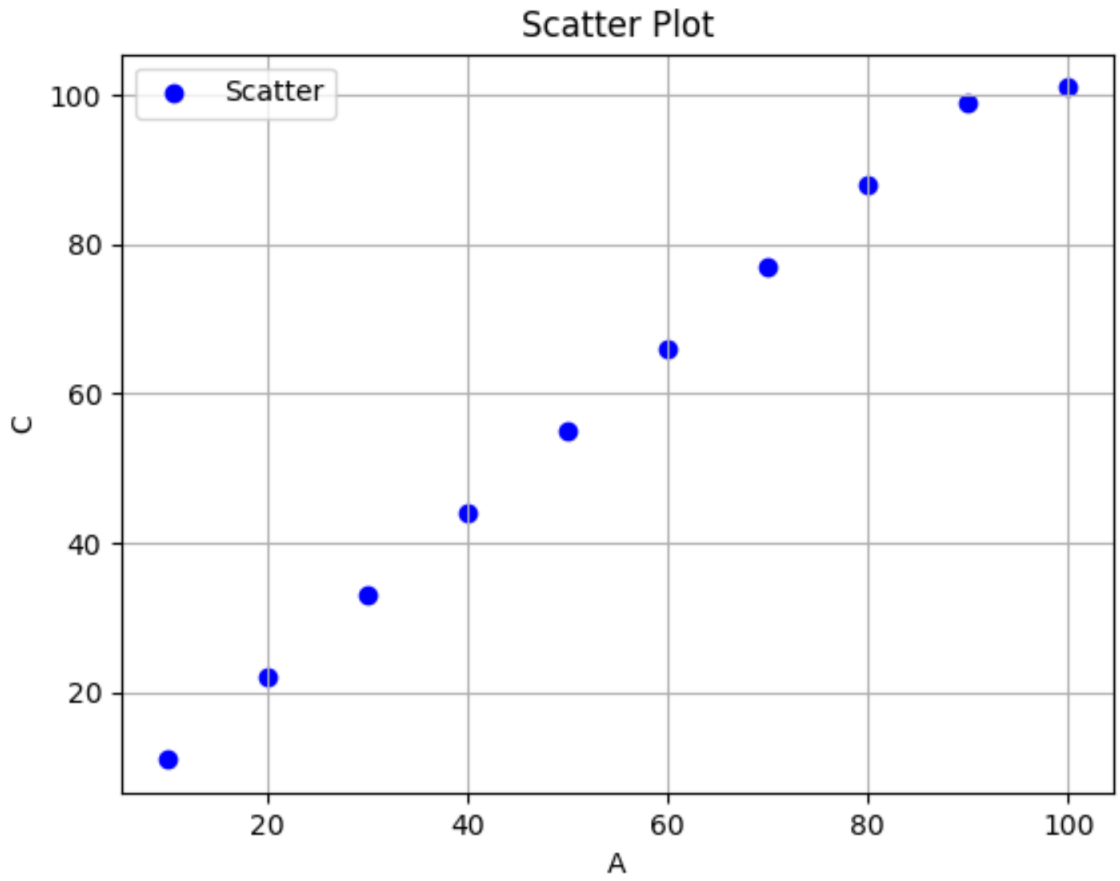
```
import pandas as pd
import matplotlib.pyplot as plt

# Given DataFrame
std_marks = {
    'A': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100],
    'B': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'C': [11, 22, 33, 44, 55, 66, 77, 88, 99, 101],
    'D': [103, 105, 109, 114, 125, 135, 149, 154, 162, 185],
    'E': [52, 86, 75, 41, 32, 71, 68, 94, 73, 69]
}

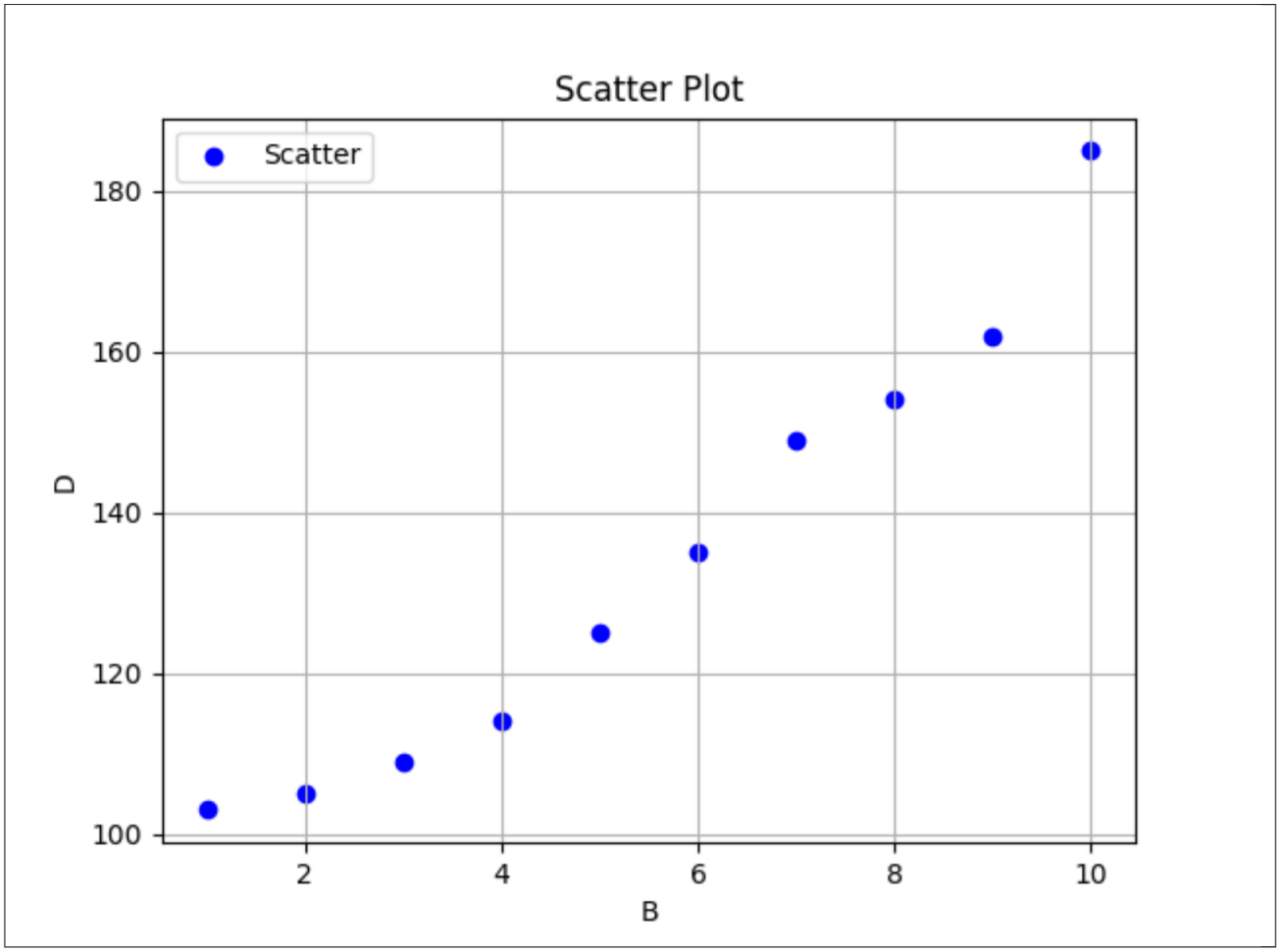
df = pd.DataFrame(std_marks)
column1, column2 = input().split()
plt.scatter(df[column1],df[column2],color='blue',marker='o',label='Scatter')
plt.xlabel(column1)
plt.ylabel(column2)
plt.title('Scatter Plot')
plt.legend()
plt.grid(True)
plt.show()
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
A C



Test Case - 2
User Output
B D


Scatter Plot
Scatter
180
160
140
120
100
D
2
4
6
8
10
B